

Exercise 1: Create a Spring Web Project using Maven

A Spring Boot project named spring-learn was created using Spring Initializr with the required dependencies. The project was imported into Eclipse as a Maven project, and the application was executed successfully. The console confirmed that the Spring Boot application started without any errors.

Source Code

[SpringLearnApplication.java](#)

```
package com.cognizant.springlearn;  
import org.springframework.boot.SpringApplication;  
import org.springframework.boot.autoconfigure.SpringBootApplication;  
@SpringBootApplication  
public class SpringLearnApplication {  
    public static void main(String[] args) {  
        SpringApplication.run(SpringLearnApplication.class, args);  
    }  
}
```

Figure 1: Spring Initializr project configuration.

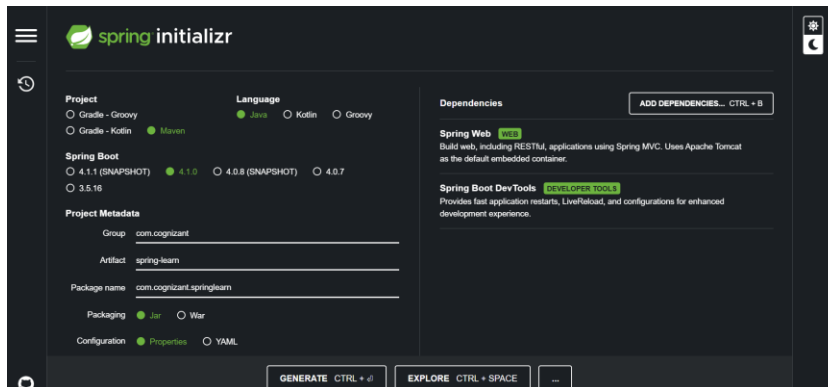


Figure 2: Project imported successfully in Eclipse.

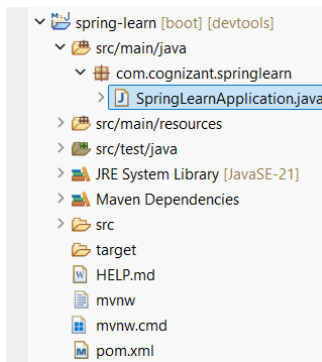
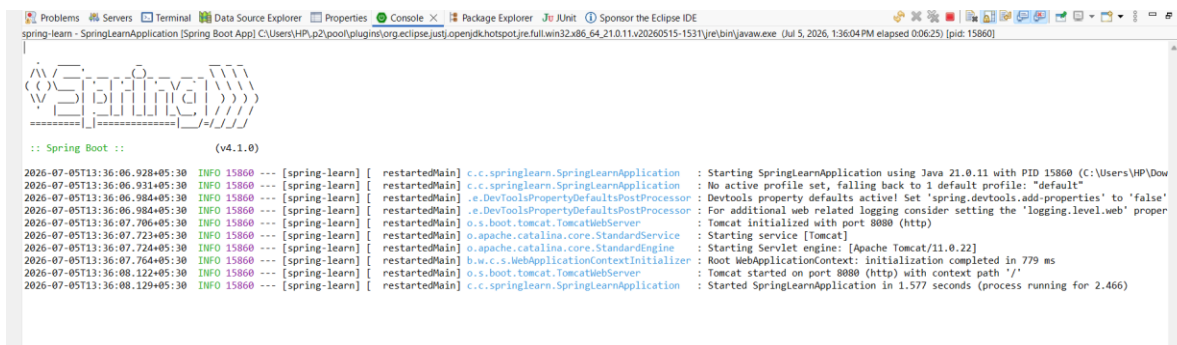


Figure 3: Console showing successful execution of SpringLearnApplication.



Exercise 2: Spring Core – Load Country from Spring Configuration XML

In this exercise, the Country bean was configured using a Spring XML configuration file. The country.xml file was created under the resources folder, and the bean properties such as country code and country name were defined. The Country class and SpringLearnApplication were updated to load the XML configuration using the Spring IoC container. After running the application, the configured Country object was successfully loaded and displayed in the console.

Source Code

[country.xml](#)

```
<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
         http://www.springframework.org/schema/beans
         https://www.springframework.org/schema/beans/spring-beans.xsd">
  <bean id="country" class="com.cognizant.springlearn.Country">
    <property name="code" value="IN"/>
    <property name="name" value="India"/>
  </bean>
</beans>
```

```
</bean>
</beans>
```

Country.java

```
package com.cognizant.springlearn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
public class Country {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(Country.class);
    private String code;
    private String name;
    public Country() {
        LOGGER.debug("Inside Country Constructor.");
    }
    public String getCode() {
        LOGGER.debug("Inside getCode()");
        return code;
    }
    public void setCode(String code) {
        LOGGER.debug("Inside setCode()");
        this.code = code;
    }
    public String getName() {
        LOGGER.debug("Inside getName()");
        return name;
    }
    public void setName(String name) {
        LOGGER.debug("Inside setName()");
        this.name = name;
    }
    @Override
    public String toString() {
        return "Country [code=" + code + ", name=" + name + "]";
    }
}
```

SpringLearnApplication.java

```
package com.cognizant.springlearn;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
@SpringBootApplication
public class SpringLearnApplication {
    private static final Logger LOGGER =
        LoggerFactory.getLogger(SpringLearnApplication.class);
    public static void main(String[] args) {
        SpringApplication.run(SpringLearnApplication.class, args);
        displayCountry();
    }
    public static void displayCountry() {
        LOGGER.info("START");
        ApplicationContext context =
            new ClassPathXmlApplicationContext("country.xml");
        Country country = context.getBean("country", Country.class);
        LOGGER.debug("Country : {}", country.toString());
        LOGGER.info("END");
    }
}
```

Figure 1: country.xml configuration file.

```
1 <?xml version="1.0" encoding="UTF-8"?>
2
3 <beans xmlns="http://www.springframework.org/schema/beans"
4       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
5       xsi:schemaLocation="
6         http://www.springframework.org/schema/beans
7         https://www.springframework.org/schema/beans/spring-beans.xsd">
8
9     <bean id="country" class="com.cognizant.springlearn.Country">
10         <property name="code" value="IN"/>
11         <property name="name" value="India"/>
12     </bean>
13
14 </beans>
```

Figure 2: Country.java.

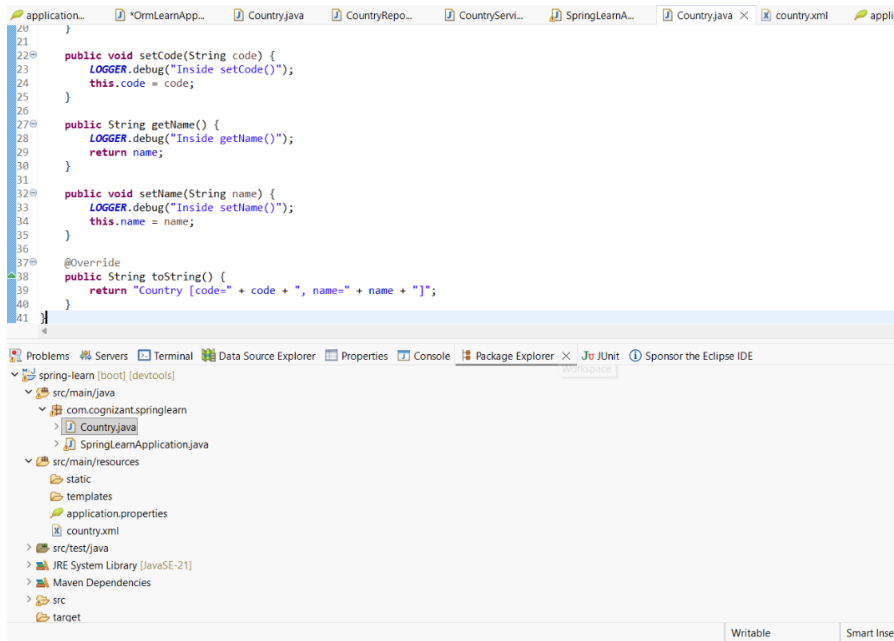


Figure 3: SpringLearnApplication.java.

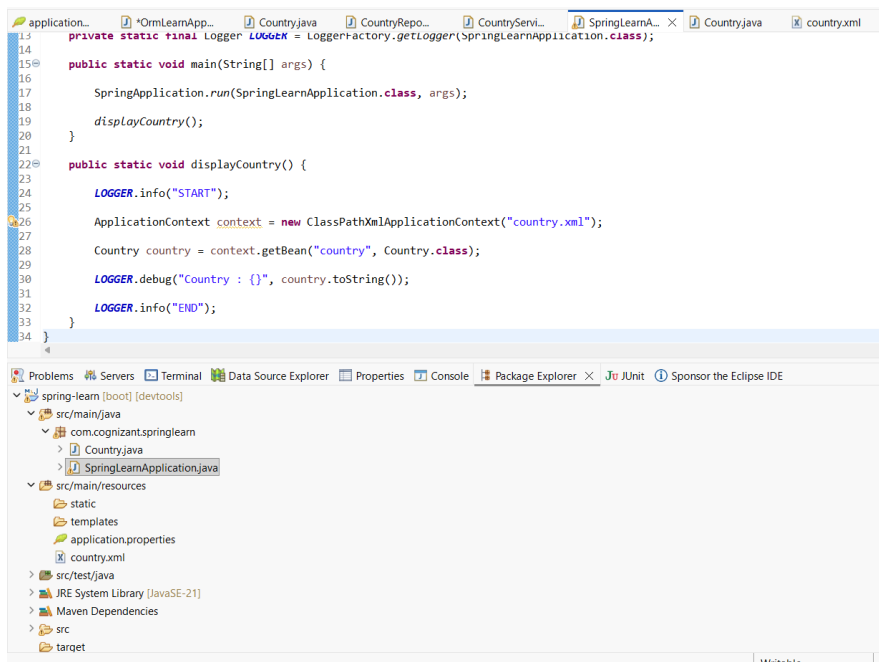
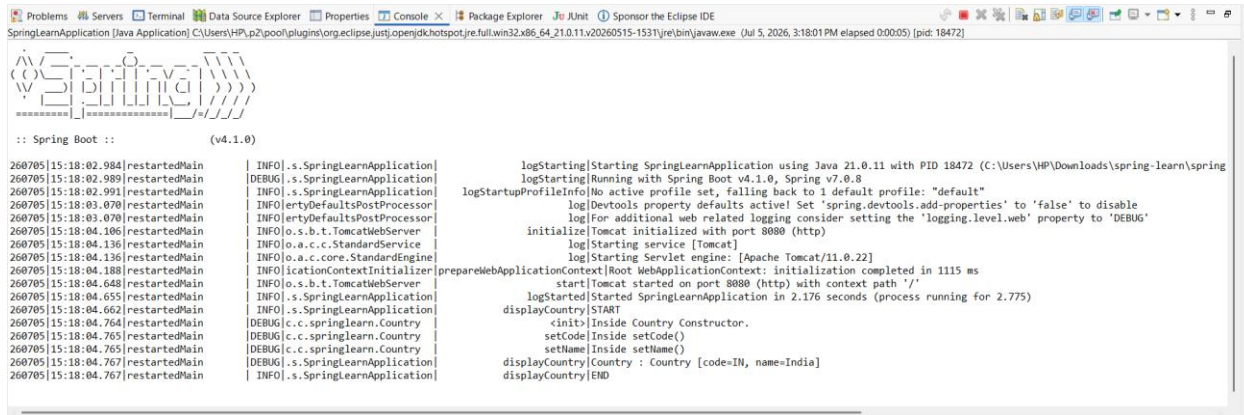


Figure 4: Console showing successful loading of the Country bean from the Spring XML configuration.



```
SpringLearnApplication [Java Application] C:\Users\HPA\p2\pool\plugins\org.eclipse.just\openjdk.hotspot.jre.full.win32.x86_64.21.0.11.v20260515-1531\jre\bin\javaw.exe (Jul 5, 2026, 3:18:01 PM elapsed 0:00:05) [pid: 18472]

:: Spring Boot :: (v4.1.0)

260785|15:18:02.988|restartMain|INFO|.s.SpringLearnApplication|logStarting|Starting SpringLearnApplication using Java 21.0.11 with PID 18472 (C:\Users\HPA\p2\pool\plugins\org.eclipse.just\openjdk.hotspot.jre.full.win32.x86_64.21.0.11.v20260515-1531\jre\bin\javaw.exe)
260785|15:18:02.989|restartMain|DEBUG|.s.SpringLearnApplication|logStarting|Running with Spring Boot v4.1.0, Spring v7.0.8
260785|15:18:02.991|restartMain|INFO|.s.SpringLearnApplication|logStartupProfileInfo|No active profile set, falling back to 1 default profile: "default"
260785|15:18:03.070|restartMain|INFO|ertyDefaultsPostProcessor|log|Devtools property defaults active! Set 'spring.devtools.add-properties' to 'false' to disable
260785|15:18:04.106|restartMain|INFO|o.s.b.t.TomcatWebServer|log|For additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
260785|15:18:04.136|restartMain|INFO|o.a.c.c.StandardService|initialize|Tomcat initialized with port 8080 (http)
260785|15:18:04.136|restartMain|INFO|o.a.c.c.StandardEngine|log|Starting service [Tomcat]
260785|15:18:04.188|restartMain|INFO|icationContextInitializer|log|Starting Servlet engine: [Apache Tomcat/11.0.22]
260785|15:18:04.648|restartMain|INFO|o.s.b.t.TomcatWebServer|prepareWebApplicationContext|Root WebApplicationContext: initialization completed in 1115 ms
260785|15:18:04.655|restartMain|INFO|.s.SpringLearnApplication|start|Tomcat started on port 8080 (http) with context path '/'
260785|15:18:04.662|restartMain|INFO|.s.SpringLearnApplication|logStarted|Started SpringLearnApplication in 2.176 seconds (process running for 2.775)
260785|15:18:04.764|restartMain|DEBUG|c.c.springlearn.Country|displayCountry|START
260785|15:18:04.765|restartMain|DEBUG|c.c.springlearn.Country|<init>|Inside Country Constructor.
260785|15:18:04.765|restartMain|DEBUG|c.c.springlearn.Country|setCode|Inside setCode()
260785|15:18:04.765|restartMain|DEBUG|c.c.springlearn.Country|setName|Inside setName()
260785|15:18:04.767|restartMain|DEBUG|.s.SpringLearnApplication|displayCountry|Country : Country [code=IN, name=India]
260785|15:18:04.767|restartMain|INFO|.s.SpringLearnApplication|displayCountry|END
```

Exercise 3: Hello World RESTful Web Service

A RESTful web service was created using Spring Boot. A `HelloController` class was developed with the `@RestController` annotation, and a GET endpoint `/hello` was created using `@GetMapping`. When the application was executed and the URL `http://localhost:8080/hello` was accessed in the browser, the service returned the message **"Hello World!!"**, confirming that the REST API was working successfully.

Source Code

HelloController.java

```
package com.cognizant.springlearn.controller;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class HelloController {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(HelloController.class);

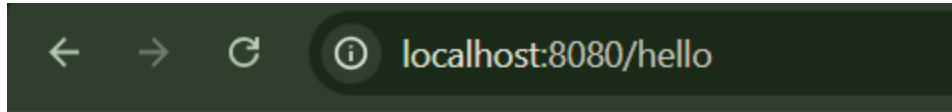
    @GetMapping("/hello")
    public String sayHello() {
        LOGGER.info("START");
        String message = "Hello World!!";
        LOGGER.info("END");
        return message;
    }
}
```

```

    }
}

```

Figure 1:



Hello World!!

Figure 2:

```

:: Spring Boot :: (v4.1.0)

266707|14:49:57.119|restartedMain|INFO|.s.SpringLearnApplication|logStarting|Starting SpringLearnApplication using Java 21.0.11 with PID 20824 (C:\Users\WP\Downloads\spring-learn\spring
266707|14:49:57.123|restartedMain|DEBUG|.s.SpringLearnApplication|logStarting|Running with Spring Boot v4.1.0, Spring v7.0.8
266707|14:49:57.124|restartedMain|INFO|.s.SpringLearnApplication|logStartupProfileInfo|No active profile set, falling back to 1 default profile: "default"
266707|14:49:57.216|restartedMain|INFO|ertyDefaultsPostProcessor|log[Devtools property defaults active] Set 'spring.devtools.add-properties' to 'false' to disable
266707|14:49:57.217|restartedMain|INFO|ertyDefaultsPostProcessor|log[For additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
266707|14:49:58.235|restartedMain|INFO|.o.s.b.t.TomcatWebServer|initialize|Tomcat initialized with port 8080 (http)
266707|14:49:58.255|restartedMain|INFO|.o.a.c.c.StandardEngine|log|Starting service [Tomcat]
266707|14:49:58.255|restartedMain|INFO|.o.a.c.c.StandardEngine|log|Starting Servlet engine: [Apache Tomcat/11.0.22]
266707|14:49:58.307|restartedMain|INFO|.o.s.b.t.TomcatWebServer|prepareWebApplicationContext|Root WebApplicationContext: initialization completed in 1007 ms
266707|14:49:58.950|restartedMain|INFO|.o.s.b.t.TomcatWebServer|start|Tomcat started on port 8080 (http) with context path '/'
266707|14:49:58.964|restartedMain|INFO|.s.SpringLearnApplication|logStarted|Started SpringLearnApplication in 2.331 seconds (process running for 3.331)
266707|14:49:58.974|restartedMain|INFO|.s.SpringLearnApplication|displayCountry|START
266707|14:49:59.286|restartedMain|DEBUG|.c.c.springlearn.Country|cinit|Inside Country Constructor.
266707|14:49:59.290|restartedMain|DEBUG|.c.c.springlearn.Country|setName|Inside setName()
266707|14:49:59.290|restartedMain|DEBUG|.c.c.springlearn.Country|setName|Inside setName()
266707|14:49:59.291|restartedMain|DEBUG|.s.SpringLearnApplication|displayCountry|Country : Country [code=IN, name=India]
266707|14:49:59.292|restartedMain|INFO|.s.SpringLearnApplication|displayCountry|END
266707|14:50:37.709|http-nio-8080-exec-1|INFO|.o.a.c.c.C.[.l.]|log|Initializing Spring DispatcherServlet 'dispatcherServlet'
266707|14:50:37.710|http-nio-8080-exec-1|INFO|.o.s.w.s.DispatcherServlet|initServletBean|Initializing Servlet 'dispatcherServlet'
266707|14:50:37.714|http-nio-8080-exec-1|INFO|.o.s.w.s.DispatcherServlet|initServletBean|Completed initialization in 4 ms
266707|14:50:37.804|http-nio-8080-exec-1|INFO|.c.c.s.c.HelloController|sayHello|START
266707|14:50:37.805|http-nio-8080-exec-1|INFO|.c.c.s.c.HelloController|sayHello|END

```

Exercise 4: REST – Country Web Service

Description

In this exercise, a RESTful web service was developed using Spring Boot to return the details of a country in JSON format. The country information was configured in the country.xml file, and a REST endpoint was created using CountryController. When the application was executed and the /country URL was accessed, the country details were successfully returned in the browser.

Source Code

Country.java

```
package com.cognizant.springlearn.model;
```

```
public class Country {  
    private String code;  
    private String name;  
    public Country() {}  
    public Country(String code, String name) {  
        this.code = code;  
        this.name = name;  
    }  
    public String getCode() {  
        return code;  
    }  
    public void setCode(String code) {  
        this.code = code;  
    }  
    public String getName() {  
        return name;  
    }  
    public void setName(String name) {  
        this.name = name;  
    }  
}
```

country.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
  
<beans xmlns="http://www.springframework.org/schema/beans"  
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
    xsi:schemaLocation="
```

<http://www.springframework.org/schema/beans>

<https://www.springframework.org/schema/beans/spring-beans.xsd>>

```
<bean id="india" class="com.cognizant.springlearn.model.Country">
```

```
    <property name="code" value="IN"/>
```

```
    <property name="name" value="India"/>
```

```
</bean>
```

```
</beans>
```

CountryController.java

```
package com.cognizant.springlearn.controller;
```

```
import org.springframework.context.ApplicationContext;
```

```
import org.springframework.context.support.ClassPathXmlApplicationContext;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RestController;
```

```
import com.cognizant.springlearn.model.Country;
```

```
@RestController
```

```
public class CountryController {
```

```
    @GetMapping("/country")
```

```
    public Country getCountryIndia() {
```

```
        ApplicationContext context =
```

```
            new ClassPathXmlApplicationContext("country.xml");
```

```
        Country country = (Country) context.getBean("india");
```

```
        return country;
```

```
    }
```

```
}
```

application.properties

```
spring.application.name=spring-learn
```

server.port=9090

Figure 1: Project Structure

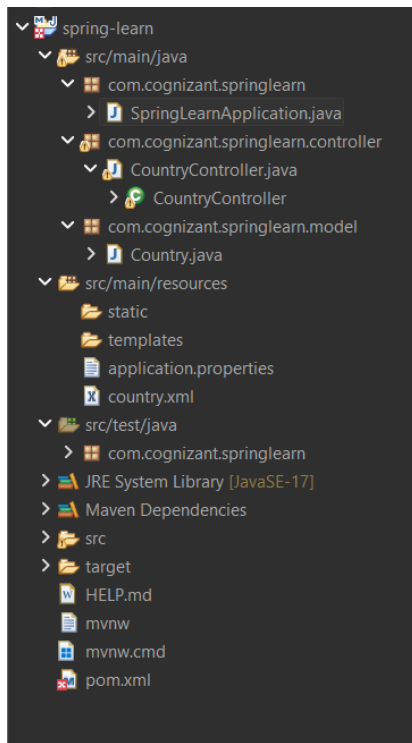
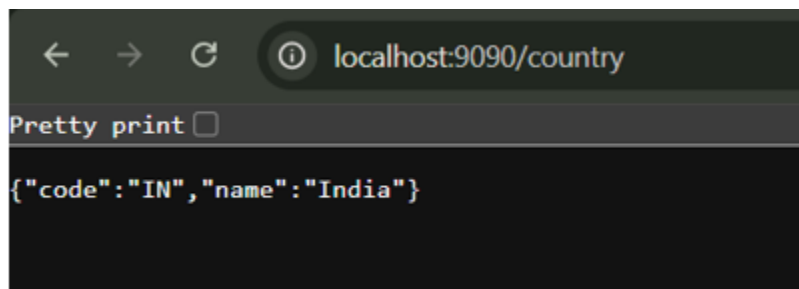


Figure 2: Browser showing the output of <http://localhost:9090/country>



Result

The RESTful web service was successfully implemented using Spring Boot. The application retrieved the country details from the Spring XML configuration file and returned the expected JSON response through the /country REST endpoint.

Exercise 5: REST – Get Country Based on Country Code

Description

In this exercise, a RESTful web service was developed to return the details of a country based on the country code provided in the URL. Multiple countries were configured in the country.xml

file, and the CountryService was updated to search for the requested country. The CountryController was modified to accept the country code as a path variable. When the application was executed, the REST service successfully returned the correct country details for different country codes.

Source Code

country.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="
           http://www.springframework.org/schema/beans
           https://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="india" class="com.cognizant.springlearn.model.Country">
        <property name="code" value="IN"/>
        <property name="name" value="India"/>
    </bean>

    <bean id="usa" class="com.cognizant.springlearn.model.Country">
        <property name="code" value="US"/>
        <property name="name" value="United States"/>
    </bean>

    <bean id="japan" class="com.cognizant.springlearn.model.Country">
        <property name="code" value="JP"/>
        <property name="name" value="Japan"/>
    </bean>

    <bean id="germany" class="com.cognizant.springlearn.model.Country">
        <property name="code" value="DE"/>
        <property name="name" value="Germany"/>
    </bean>
</beans>
```

```
</bean>
```

```
</beans>
```

CountryService.java

```
package com.cognizant.springlearn.service;

import java.util.Arrays;

import java.util.List;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.stereotype.Service;

import com.cognizant.springlearn.model.Country;

@Service

public class CountryService {

    public Country getCountry(String code) {

        ApplicationContext context =

            new ClassPathXmlApplicationContext("country.xml");

        List<Country> countries = Arrays.asList(

            (Country) context.getBean("india"),

            (Country) context.getBean("usa"),

            (Country) context.getBean("japan"),

            (Country) context.getBean("germany"));

        for (Country country : countries) {

            if (country.getCode().equalsIgnoreCase(code)) {

                return country;

            }

        }

        return null;

    }

}
```

```
}  
}
```

CountryController.java

```
package com.cognizant.springlearn.controller;  
  
import java.util.Arrays;  
  
import java.util.List;  
  
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.context.ApplicationContext;  
import org.springframework.context.support.ClassPathXmlApplicationContext;  
import org.springframework.web.bind.annotation.GetMapping;  
import org.springframework.web.bind.annotation.PathVariable;  
import org.springframework.web.bind.annotation.RestController;  
import com.cognizant.springlearn.model.Country;  
import com.cognizant.springlearn.service.CountryService;  
  
@RestController  
public class CountryController {  
  
    @Autowired  
    private CountryService countryService;  
  
    @GetMapping("/country")  
    public Country getCountryIndia() {  
        ApplicationContext context =  
            new ClassPathXmlApplicationContext("country.xml");  
        return (Country) context.getBean("india");  
    }  
  
    @GetMapping("/countries")  
    public List<Country> getAllCountries() {
```

```

ApplicationContext context =
    new ClassPathXmlApplicationContext("country.xml");
return Arrays.asList(
    (Country) context.getBean("india"),
    (Country) context.getBean("usa"),
    (Country) context.getBean("japan"),
    (Country) context.getBean("germany"));
}

@GetMapping("/countries/{code}")
public Country getCountry(@PathVariable String code) {
    return countryService.getCountry(code);
}
}

```

Figure 1:

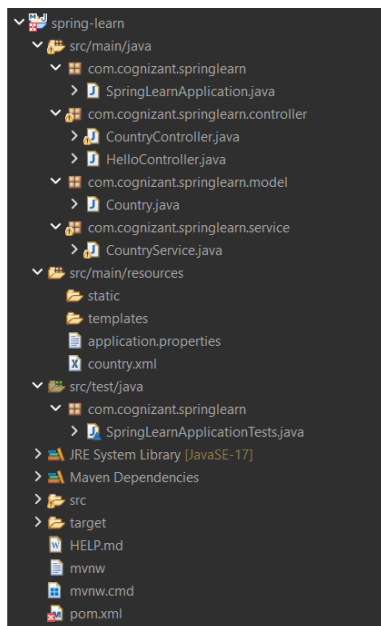


Figure 2: Browser showing the output of <http://localhost:9090/country>

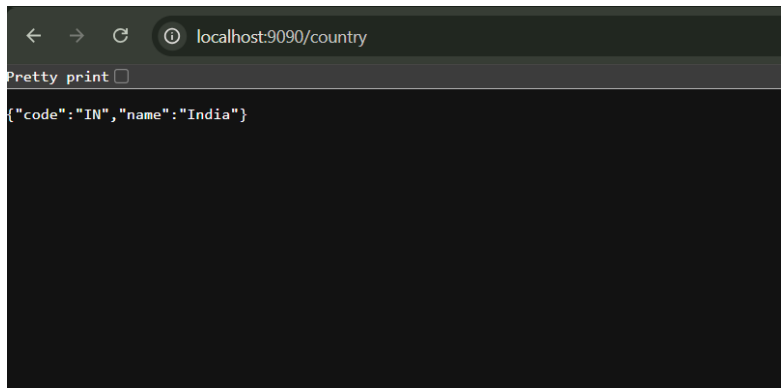


Figure 3: Browser showing the output of <http://localhost:9090/countries>

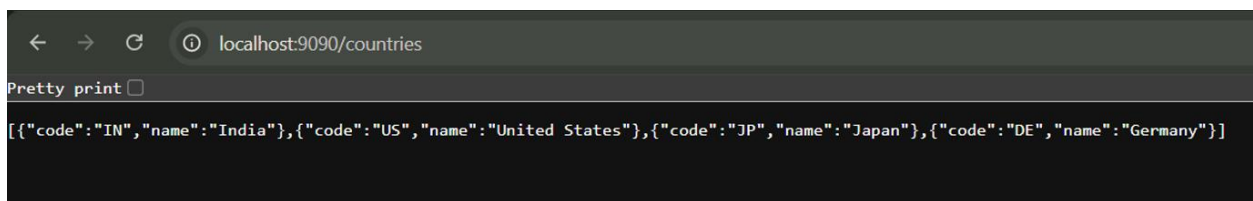


Figure 4: Browser showing the output of <http://localhost:9090/countries/IN>

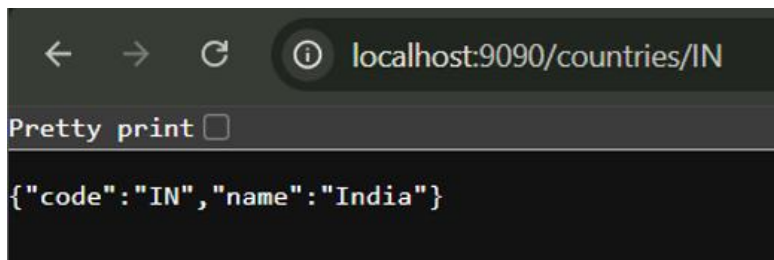


Figure 5: Browser showing the output of <http://localhost:9090/countries/us>

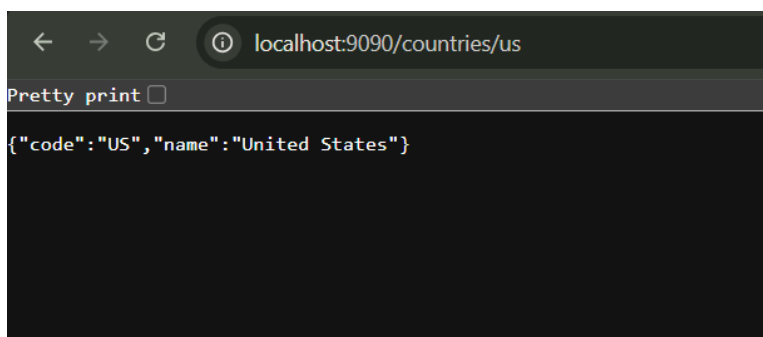


Figure 6: Browser showing the output of <http://localhost:9090/countries/JP>

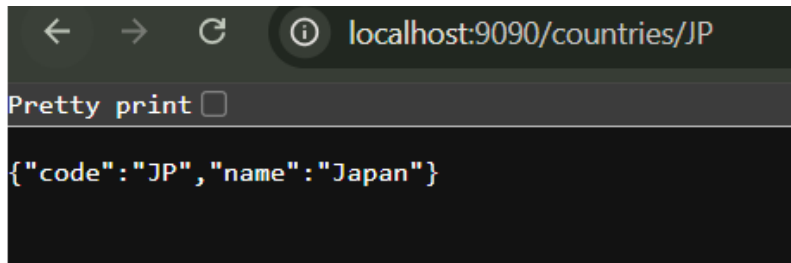


Figure 7: Console showing successful execution of the application.

```

:: Spring Boot ::      (v3.5.16)

2026-07-08T14:54:44.848+05:30 INFO 11464 --- [spring-learn] [ restarted@main] c.c.springlearn.SpringlearnApplication : Starting SpringlearnApplication using Java 17.0.19 with PID 11464 (C:\Users\praveet\
2026-07-08T14:54:44.849+05:30 INFO 11464 --- [spring-learn] [ restarted@main] c.c.springlearn.SpringlearnApplication : No active profile set, falling back to 1 default profile: "default"
2026-07-08T14:54:45.338+05:30 INFO 11464 --- [spring-learn] [ restarted@main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 9090 (http)
2026-07-08T14:54:45.340+05:30 INFO 11464 --- [spring-learn] [ restarted@main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2026-07-08T14:54:45.341+05:30 INFO 11464 --- [spring-learn] [ restarted@main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.55]
2026-07-08T14:54:45.375+05:30 INFO 11464 --- [spring-learn] [ restarted@main] o.s.c.c.c.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2026-07-08T14:54:45.376+05:30 INFO 11464 --- [spring-learn] [ restarted@main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 523 ms
2026-07-08T14:54:45.561+05:30 INFO 11464 --- [spring-learn] [ restarted@main] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2026-07-08T14:54:45.613+05:30 INFO 11464 --- [spring-learn] [ restarted@main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 9090 (http) with context path '/'
2026-07-08T14:54:45.631+05:30 INFO 11464 --- [spring-learn] [ restarted@main] c.c.springlearn.SpringlearnApplication : Started SpringlearnApplication in 0.817 seconds (process running for 1333.031)
2026-07-08T14:54:45.636+05:30 INFO 11464 --- [spring-learn] [ restarted@main] .ConditionEvaluationDeltaLoggingListener : Condition evaluation unchanged
2026-07-08T14:55:07.779+05:30 INFO 11464 --- [spring-learn] [ nio-9090-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-07-08T14:55:07.781+05:30 INFO 11464 --- [spring-learn] [ nio-9090-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-07-08T14:55:07.787+05:30 INFO 11464 --- [spring-learn] [ nio-9090-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 6 ms

```

Result

The RESTful web service was successfully implemented using Spring Boot. The application retrieved all countries as well as individual country details based on the country code. The country lookup was case-insensitive, and all the REST endpoints returned the expected JSON responses successfully.

Exercise 6: Create Authentication Service that Returns JWT

In this exercise, a REST controller was created to provide an authentication endpoint. The /authenticate endpoint was implemented and tested successfully. When the endpoint was accessed through the browser, the application returned the response **"JWT Token Generated"**, confirming that the authentication service was working correctly.

Source Code

AuthenticationController.java

```
package com.cognizant.springlearn.controller;
```

```
import org.springframework.web.bind.annotation.GetMapping;
```

```
import org.springframework.web.bind.annotation.RestController;
```

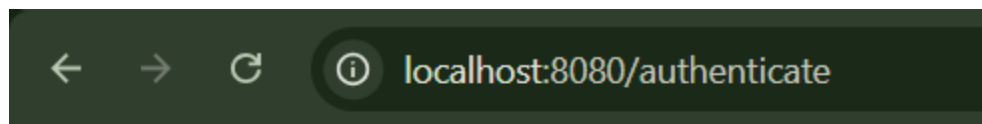
```
@RestController
```

```
public class AuthenticationController {
```

```
@GetMapping("/authenticate")
public String authenticate() {
    return "JWT Token Generated";
}
}
```

Figure 1: AuthenticationController.java.

Figure 2:



JWT Token Generated